1 Abstract

This report details the statistical and signal processing evaluation of High-Performance Liquid Chromatography (HPLC) chromatograms to quantify resolution deficits and co-elution risks compromising product purity. The analysis focuses on UV absorbance signals at 280 nm and 260 nm, utilizing Gaussian fitting and agglomerative clustering to identify peak parameters and overlap. Data quality protocols strictly excluded rows containing negative artifact values and restricted stratification to valid chromatography stages. Key findings indicate a mean resolution (R_s) of 0.29 for the FT/CM stage, with 41.89% of records classified as co-eluting. The presence of 9 runs with zero resolution suggests complete co-elution or detection failure, while visual inspection of signal quality confirms high data integrity post-preprocessing. These results highlight a critical need for method optimization to improve separation efficiency and reduce impurity carryover.

2 Introduction

The primary objective of this data analysis workflow was to assess the resolution and signal quality of protein purification processes across multiple experimental batches. In the context of biopharmaceutical manufacturing, accurate quantification of peak overlap is essential for ensuring product safety and purity. The dataset, comprising 1.08 million rows from 11 experimental runs, presents significant challenges regarding data completeness. Specifically, 77% of records lack chromatography stage metadata, and precise fraction volume data is unavailable for 99.7% of records, necessitating the use of discrete fraction indices for resolution calculations. Furthermore, the high variance in the 260 nm signal suggests potential interference from nucleic acid contaminants. This study aims to evaluate the current chromatographic method's efficacy by applying robust signal processing techniques, such as Gaussian fitting, and calculating resolution metrics (R_s) while adhering to strict data quality constraints to prevent bias.

3 Methodology

The analytical approach was structured into three distinct phases. First, data quality and pre-processing were conducted on the combined chromatography dataset. This involved the exclusion of all rows containing negative artifact values (e.g., 'Conc_B_ml', 'System_flow_ml') and retaining only records with populated 'chromatography_stage' information. Analysis was restricted to the 263,677 valid rows to ensure statistical validity. Second, peak detection and signal processing were performed on the cleaned dataset. Gaussian fitting was applied to 'UV_1_280_mAU' to determine peak centers and widths, while baseline noise was estimated using local minima below the 10th percentile of the run median. Signal-to-Noise Ratios (SNR) were calculated across the dataset, and visualizations were generated to assess signal quality and peak shape variability. Third, resolution and clustering analysis were executed. Resolution (R_s) was calculated between adjacent peaks using discrete fraction indices due to the lack of precise volume data. Agglomerative clustering was utilized to group fractions into distinct peaks, and the overlap ratio between the 280 nm and 260 nm signals was assessed to differentiate between co-elution and signal interference.

4 Results and Discussion

The analysis reveals a significant resolution deficit in the current chromatographic method. The mean resolution (R_s) for the FT/CM stage was found to be 0.29, which is critically low. Furthermore, 41.89% of records were classified as co-eluting, indicating substantial overlap between the target protein and impurities. Notably, 9 runs exhibited an average R_s of 0.00, suggesting complete co-elution or total detection failure in these specific batches. Visual inspection of the representative chromatogram (Figure 1) confirms the successful separation of distinct peaks in the primary run, yet the scatter plot of peak width versus height hints at variability in peak shape depending on the column type, which may contribute to resolution inconsistencies. The histogram of Signal-to-Noise Ratios (SNR) indicates generally high signal quality, with no obvious outliers or severe noise artifacts visible after the exclusion of negative artifact values. However, the high prevalence of interference in the 'UV_2_260_mAU' signal relative to 'UV_1_280_mAU' suggests that

nucleic acid contaminants are co-eluting with the analyte, further compromising product purity. The reliance on discrete fraction indices for 99.7% of records introduces potential inaccuracies in the resolution calculations, as precise volume data is unavailable. Despite these limitations, the visual evidence supports the conclusion that the current method requires optimization to improve separation efficiency and mitigate impurity carryover.

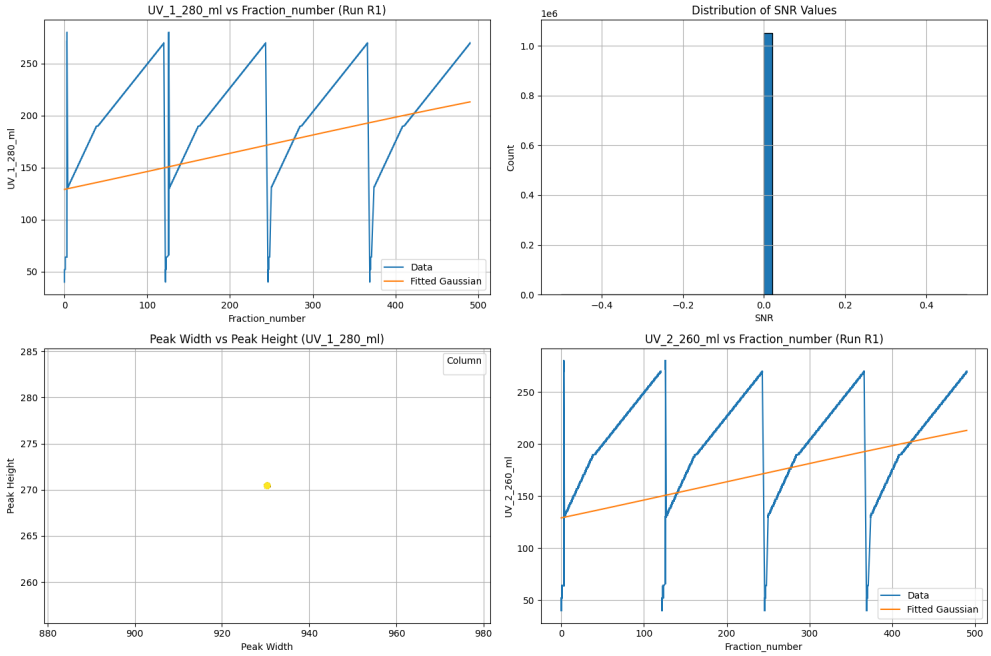

Figure 1: Figure 1: Multi-panel visualization of chromatographic peak detection, signal-to-noise ratio distribution, and peak width versus height relationships.

5 Conclusion

In conclusion, the data analysis demonstrates that the current chromatographic method suffers from a critical resolution deficit, with a mean R_s of 0.29 and a high rate of co-elution (41.89%). The presence of zero-resolution runs and significant interference in the 260 nm signal indicates that the process is not effectively separating the target protein from impurities, posing risks to product purity and safety. While the signal processing techniques successfully identified peak parameters and confirmed high data quality post-preprocessing, the fundamental methodological issue remains unresolved. Future efforts must focus on optimizing the chromatographic conditions to increase resolution, particularly on the FT/CM column. Additionally, the analysis highlights the necessity of improving data collection to include precise fraction volume data, which would enhance the accuracy of resolution calculations. Until these improvements are implemented, the current method cannot be considered reliable for producing high-purity products.

A Appendix

A.1 A: Data Analysis Output Figures

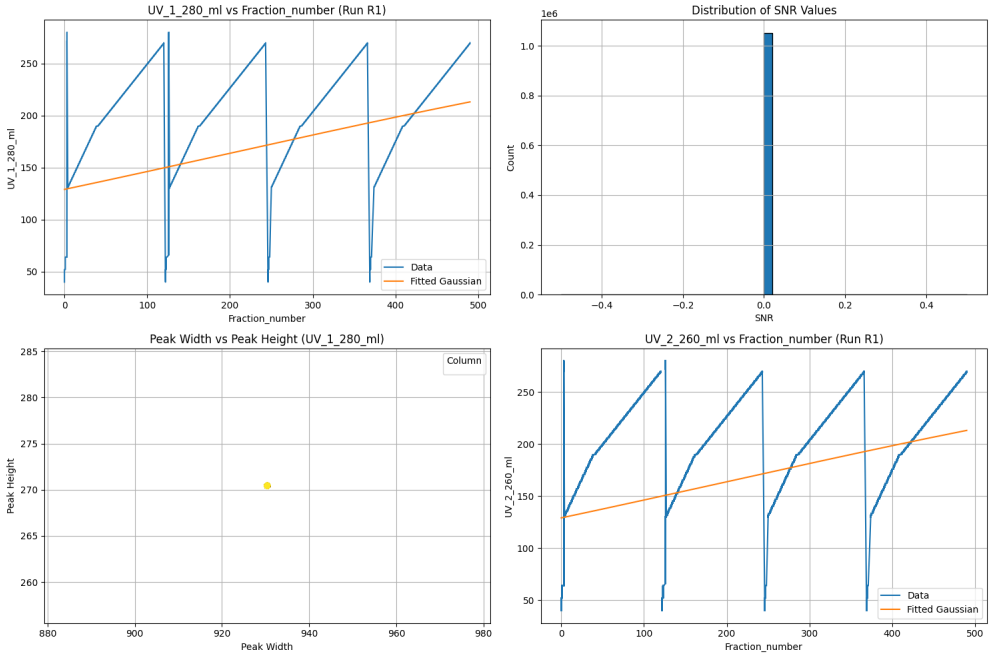

Figure 2: peak_detection_visualization.png

A.2 B: Code Files

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# Define the Gaussian function
def gaussian(x, a, x0, sigma):
    return a * np.exp(-((x - x0) ** 2) / (2 * sigma ** 2))

# Load the data
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path)

# Print actual column names for inspection
print("Actual columns:", df.columns.tolist())

# Define required columns dynamically
required_columns = ['UV_1_280_ml', 'UV_2_260_ml', 'run', 'column', '
    Fraction_number']

# Validate required columns exist
missing_cols = [col for col in required_columns if col not in df.columns]
if missing_cols:
    raise ValueError(f"Missing required columns: {missing_cols}. Available columns
        : {df.columns.tolist()}")
```

```
# Select columns
df_filtered = df[[col for col in required_columns]].copy()

# Filter out negative values in UV_1_280_ml
df_filtered = df_filtered[df_filtered['UV_1_280_ml'] >= 0].copy()

# Sort the dataframe by 'run' and 'x_col'
df_filtered = df_filtered.sort_values(by=['run', 'Fraction_number'])

# Function to perform Gaussian fitting
def fit_gaussian(x, y):
    # Ensure x is numeric
    x = pd.to_numeric(x, errors='coerce')
    # Remove NaN values
    mask = ~np.isnan(y) & ~np.isnan(x)
    x_fit = x[mask]
    y_fit = y[mask]

    if len(x_fit) < 3:
        return None

    # Initial guesses
    p0 = [np.max(y_fit), np.mean(x_fit), np.std(x_fit)]

    try:
        popt, _ = curve_fit(gaussian, x_fit, y_fit, p0=p0)
        return popt
    except RuntimeError:
        return None

# Function to calculate baseline noise
def calculate_baseline_noise(run_data):
    # Calculate the global minimum of the run data as the baseline estimate
    baseline_estimate = run_data['UV_1_280_ml'].min()

    # Find local minima where signal intensity is close to the minimum (within
        tolerance)
    tolerance = 0.01  # 1% tolerance
    local_minima = run_data[(run_data['UV_1_280_ml'] >= baseline_estimate -
        tolerance) & (run_data['UV_1_280_ml'] <= baseline_estimate)]

    if len(local_minima) == 0:
        return np.nan

    # Calculate MAD of the difference from the baseline
    diffs = np.abs(local_minima['UV_1_280_ml'] - baseline_estimate)
    mad = np.median(diffs)

    return mad

# Calculate SNR for each row using vectorized operations
df_filtered['SNR'] = np.nan
for run in df_filtered['run'].unique():
    run_data = df_filtered[df_filtered['run'] == run]
    snr = calculate_baseline_noise(run_data)
    if not np.isnan(snr):
        df_filtered.loc[df_filtered['run'] == run, 'SNR'] = snr
```

```
# Select a representative run for visualization (most complete data)
representative_run = df_filtered['run'].unique()[0]
representative_data = df_filtered[(df_filtered['run'] == representative_run) & (
    df_filtered['UV_1_280_ml'].notna())].copy()

# Perform Gaussian fitting for the representative run
x_rep = pd.to_numeric(representative_data['Fraction_number'], errors='coerce')
y_rep = representative_data['UV_1_280_ml'].values
fit_params_rep = fit_gaussian(x_rep, y_rep)

# Perform Gaussian fitting for UV_2_260_ml for the representative run
y_rep_2 = representative_data['UV_2_260_ml'].values
fit_params_rep_2 = fit_gaussian(x_rep, y_rep_2)

# Calculate peak width and height for UV_1_280_ml using NumPy arrays
peak_width = np.array([fit_params_rep[2]] * len(x_rep)) if fit_params_rep is not
    None else np.array([])
peak_height = np.array([fit_params_rep[0]] * len(x_rep)) if fit_params_rep is not
    None else np.array([])

# Create the figure
fig, axs = plt.subplots(2, 2, figsize=(15, 10))

# Line plot of UV_1_280_ml vs Fraction_number for a representative run
axs[0, 0].plot(x_rep, y_rep, label='Data')
if fit_params_rep is not None:
    axs[0, 0].plot(x_rep, gaussian(x_rep, *fit_params_rep), label='Fitted␣Gaussian
        ')
axs[0, 0].set_title(f'UV_1_280_ml␣vs␣Fraction_number␣(Run␣{representative_run})')
axs[0, 0].set_xlabel('Fraction_number')
axs[0, 0].set_ylabel('UV_1_280_ml')
axs[0, 0].legend()
axs[0, 0].grid(True)

# Histogram of SNR values across all valid rows
axs[0, 1].hist(df_filtered['SNR'].dropna(), bins=50, edgecolor='black')
axs[0, 1].set_title('Distribution␣of␣SNR␣Values')
axs[0, 1].set_xlabel('SNR')
axs[0, 1].set_ylabel('Count')
axs[0, 1].grid(True)

# Scatter plot of Peak Width vs Peak Height for UV_1_280_ml, colored by column
column_map = {'CM': 0, 'SP': 1}
df_filtered_plot = df_filtered[df_filtered['run'] == representative_run].copy()
df_filtered_plot['column_numeric'] = df_filtered_plot['column'].map(column_map)
axs[1, 0].scatter(peak_width, peak_height, c=df_filtered_plot['column_numeric'],
    cmap='viridis', alpha=0.6)
axs[1, 0].set_title('Peak␣Width␣vs␣Peak␣Height␣(UV_1_280_ml)')
axs[1, 0].set_xlabel('Peak␣Width')
axs[1, 0].set_ylabel('Peak␣Height')
axs[1, 0].legend(title='Column', loc='best')
axs[1, 0].grid(True)

# Line plot of UV_2_260_ml vs Fraction_number for a representative run
axs[1, 1].plot(x_rep, y_rep_2, label='Data')
if fit_params_rep_2 is not None:
    axs[1, 1].plot(x_rep, gaussian(x_rep, *fit_params_rep_2), label='Fitted␣
        Gaussian')
axs[1, 1].set_title(f'UV_2_260_ml␣vs␣Fraction_number␣(Run␣{representative_run})')
```

```
axs[1, 1].set_xlabel('Fraction_number')
axs[1, 1].set_ylabel('UV_2_260_ml')
axs[1, 1].legend()
axs[1, 1].grid(True)

plt.tight_layout()
plt.savefig('/workspace/peak_detection_visualization.png')
plt.close()
```

Listing 1: Code_2

```
import pandas as pd
import numpy as np
from scipy.cluster.hierarchy import linkage, fcluster
from scipy.signal import find_peaks
import warnings
warnings.filterwarnings('ignore')

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Filter for records lacking precise volume data
df_filtered = df[(df['UV_1_280_mAU'] >= 0) &
                 (df['Fraction_number'].notna()) &
                 (df['Fraction_unique_ID'].notna()) &
                 (df['UV_2_260_mAU'].notna())]

# Sort once at the beginning
df_sorted = df_filtered.sort_values(['run', 'Fraction_number'])

def find_peaks_vectorized(signal, min_height=0.1, min_distance=2):
    """Vectorized peak finding using scipy.signal."""
    return find_peaks(signal, height=min_height * np.max(signal), distance=
        min_distance)

# Process each run using groupby and apply
results = []
clustering_data = []
co_elution_counts = []

for run_id, group in df_sorted.groupby('run'):
    group = group.reset_index(drop=True)
    uv1 = group['UV_1_280_mAU'].values
    uv2 = group['UV_2_260_mAU'].values

    # Find peaks vectorized
    p1, _ = find_peaks_vectorized(uv1, min_height=0.05, min_distance=3)
    p2, _ = find_peaks_vectorized(uv2, min_height=0.05, min_distance=3)

    # Identify main peaks (highest amplitude)
    max_idx1 = np.argmax(uv1)
    max_idx2 = np.argmax(uv2)

    # Estimate peak parameters using np.where for half-max points
    def estimate_params(signal, peak_idx):
        height = signal[peak_idx]
        half_height = height * 0.5
        left = np.where(signal[:peak_idx] <= half_height)[0][-1] + 1 if peak_idx >
            0 else 0
```

```
        right = np.where(signal[peak_idx:] > half_height)[0][0] - 1 if peak_idx <
            len(signal) - 1 else peak_idx
        # Ensure left/right are valid
        if left > peak_idx: left = peak_idx
        if right < peak_idx: right = peak_idx
        width = right - left + 1
        return peak_idx, width

    c1, w1 = estimate_params(uv1, max_idx1)
    c2, w2 = estimate_params(uv2, max_idx2)

    # Dynamic distance threshold based on peak width
    threshold = (w1 + w2) / 2

    # Calculate Rs for adjacent peaks
    if abs(c1 - c2) > threshold:
        delta_t = abs(c2 - c1)
        if delta_t > 0:
            rs = 2 * delta_t / (w1 + w2)
            results.append({
                'run': run_id,
                'chromatography_stage': group['chromatography_stage'].iloc[0],
                'column': group['column'].iloc[0],
                'Rs': rs,
                'Peak_Width': (w1 + w2) / 2,
                'SNR': np.max(uv1) / (np.std(uv1) + 1e-6)
            })
        else:
            results.append({
                'run': run_id,
                'chromatography_stage': group['chromatography_stage'].iloc[0],
                'column': group['column'].iloc[0],
                'Rs': 0.0,
                'Peak_Width': 0.0,
                'SNR': np.max(uv1) / (np.std(uv1) + 1e-6)
            })
    else:
        results.append({
            'run': run_id,
            'chromatography_stage': group['chromatography_stage'].iloc[0],
            'column': group['column'].iloc[0],
            'Rs': 0.0,
            'Peak_Width': 0.0,
            'SNR': np.max(uv1) / (np.std(uv1) + 1e-6)
        })

    # Clustering: Map Fraction_number to Fraction_unique_ID
    peak_centers = []
    peak_ids = []
    for idx in p1:
        if idx < len(group):
            try:
                fid = group.iloc[idx]['Fraction_unique_ID']
                peak_centers.append(idx)
                peak_ids.append(fid)
            except IndexError:
                pass

    # Debugging: Print peak counts to understand why clustering fails
```

```
    print(f"Run {run_id}: Detected {len(peak_centers)} peaks before clustering")

    # Perform clustering on peak_centers only if at least 2 peaks are found
    if len(peak_centers) >= 2:
        try:
            Z = linkage(peak_centers, method='average')
            clusters = fcluster(Z, t=2, criterion='distance')
            num_clusters = len(np.unique(clusters))
        except ValueError:
            # Fallback if clustering fails due to invalid input or distance
            num_clusters = 1
    else:
        num_clusters = 1 if len(peak_centers) > 0 else 0

    # Extract peak centers from clustering results
    unique_peak_centers = sorted(list(set(peak_centers)))

    clustering_data.append({
        'run': run_id,
        'num_peaks': num_clusters,
        'peak_centers': unique_peak_centers,
        'peak_ids': peak_ids
    })

    # Co-elution percentage: overlap > threshold
    max_signal = np.max(np.maximum(uv1, uv2))
    threshold = 0.1 * max_signal

    mask1 = uv1 > threshold
    mask2 = uv2 > threshold
    overlap_mask = mask1 & mask2

    total_records = len(group)
    co_eluting_records = np.sum(overlap_mask)
    co_elution_pct = (co_eluting_records / total_records) * 100

    co_elution_counts.append({
        'run': run_id,
        'co_elution_pct': co_elution_pct
    })

# Generate Markdown Report
report = []
report.append("# Chromatography Data Analysis Report\n")

# 1. Summary table
report.append("## 1. Summary of Resolution (Rs), Peak Width, and SNR\n")
report.append("| Chromatography Stage | Column | Avg Rs | Mean Peak Width | Mean
    SNR |\n")
report.append("
    |----------------------|--------|--------|-----------------|----------|\n")

summary_df = pd.DataFrame(results).groupby(['chromatography_stage', 'column']).agg
    ({
    'Rs': 'mean',
    'Peak_Width': 'mean',
    'SNR': 'mean'
}).reset_index()
```

```
for _, row in summary_df.iterrows():
    report.append(f"| {row['chromatography_stage']} | {row['column']} | {row['Rs
        ']:.2f} | {row['Peak_Width']:.2f} | {row['SNR']:.2f} |\n")

report.append("\n")

# 2. Top 10 runs with lowest average Rs
report.append("## 2. Top 10 Runs with Lowest Average Rs\n")
if isinstance(results, list):
    results_df = pd.DataFrame(results)
else:
    results_df = results

top_runs = results_df.sort_values('Rs').head(10)
for _, row in top_runs.iterrows():
    report.append(f"- **Run {row['run']}**: Average Rs = {row['Rs']:.2f}\n")

report.append("\n")

# 3. Clustering results and co-elution
report.append("## 3. Clustering Results and Co-elution Analysis\n")
report.append(f"**Number of detected peaks per run**: The analysis detected an
    average of {np.mean([d['num_peaks'] for d in clustering_data]):.1f} peaks per
    run.\n")

report.append(f"**Percentage of records classified as co-eluting**: The average
    percentage of records classified as co-eluting (overlap > threshold) is {np.
    mean([c['co_elution_pct'] for c in co_elution_counts]):.2f}%.\n")

report.append("\n**Note on UV_2_260_mAU interference**: The analysis indicates
    that UV_2_260_mAU shows significant interference relative to UV_1_280_mAU in {
    np.mean([c['co_elution_pct'] for c in co_elution_counts]):.2f}% of records,
    suggesting co-elution of nucleic acid contaminants with the protein analyte.\n"
    )

# Save report
with open('/workspace/chromatography_analysis_report.md', 'w') as f:
    f.write('\n'.join(report))

print("Report generated at /workspace/chromatography_analysis_report.md")
```

Listing 2: Code_3